

=====

DOCUMENTACAO DO SISTEMA - POO PUC MINAS

=====

Data de geracao: 03/06/2026

Repositorio: /home/Ale/projects/PUCMINAS/POO

Disciplina: Programacao Orientada a Objetos (POO)

Instituicao: Pontificia Universidade Catolica de Minas Gerais (PUC Minas)

Tecnologia: C# / .NET 8.0

=====

1. VISAO GERAL DO PROJETO

O repositorio contem o seguinte sub-projeto academico:

- trabalhopratiko/ : Trabalho pratico principal

Sistema de Gerenciamento de Biblioteca (SistemaBiblioteca)

Aplicacao console que demonstra conceitos avancados de POO:

heranca multinivel, encapsulamento com validacao, interfaces

genericas e gerenciamento CRUD em memoria.

2. ESTRUTURA DE DIRETORIOS

POO/

└─ README.md

└─ sistema.txt (este arquivo)

|

└─ trabalhopratico/

└─ SistemaBiblioteca.csproj

└─ program.cs

└─ Interfaces/

| └─ IRepository.cs

└─ Models/

| └─ Pessoa.cs

| └─ Usuario.cs

| └─ Funcionario.cs

| └─ Bibliotecario.cs

| └─ Aluno.cs

| └─ ItemBiblioteca.cs

| └─ Livro.cs

| └─ Emprestimo.cs

└─ Menu/

| └─ MenuPrincipal.cs

└─ Utils/

└─ Validador.cs

=====

=====

TRABALHO PRATICO: SistemaBiblioteca

=====

Aplicacao console .NET 8.0 de gerenciamento de biblioteca.

Ponto de entrada: program.cs -> MenuPrincipal.Iniciar()

3. HIERARQUIA DE CLASSES — PESSOAS

Pessoa (abstrata)

└─ Usuario

 └─ Funcionario

 └─ Bibliotecario

└─ Aluno (herda diretamente de Pessoa)

CLASSE ABSTRATA: Pessoa

Namespace: SistemaBiblioteca.Models

Arquivo: Models/Pessoa.cs

Campos privados (backing fields):

_id, _nome, _cpf, _dataNascimento

Propriedades:

Id : int (deve ser > 0)

Nome : string (minimo 3 chars, trim)
Cpf : string (11 digitos numericos, sem formatacao)
DataNascimento : DateTime (entre 01/01/1900 e hoje)

Propriedades computadas:

CpfFormatado : string ("XXX.XXX.XXX-XX")
Idade : int (anos completos ate hoje)

Metodos:

ObterTipo() : abstract string (retorna label do tipo)
ObterResumo() : virtual string (resumo formatado)
ToString() : override (chama ObterResumo())

CLASSE: Usuario (herda Pessoa)

Arquivo: Models/Usuario.cs

Propriedades adicionais:

Email : string (deve conter '@' e '.'; lowercase, trim)
Telefone : string (10 ou 11 digitos, strip formatacao)
DataCadastro : DateTime (set privado; DateTime.Now na construcao)
Ativo : bool

Propriedades computadas:

TelefoneFormatado : string ("(XX) XXXXX-XXXX" ou "(XX) XXXX-XXXX")

Metodos sobrescritos:

ObterTipo() -> "Usuario"

ObterResumo() -> estende base com email, telefone, status

CLASSE: Funcionario (herda Usuario)

Arquivo: Models/Funcionario.cs

Propriedades adicionais:

Matricula : string (uppercase, trim)

Cargo : string

Salario : decimal (≥ 0)

DataAdmissao : DateTime

Propriedades computadas:

AnosServico : int (anos desde DataAdmissao)

Metodos sobrescritos:

ObterTipo() -> "Funcionario"

ObterResumo() -> adiciona matricula, cargo, data de admissao

Metodo virtual:

ObterPermissoes() -> string com permissoes base do funcionario

CLASSE: Bibliotecario (herda Funcionario)

Arquivo: Models/Bibliotecario.cs

Propriedades adicionais:

Crb : string (registro CRB)

ChefeDivisao : bool

Especialidades : IReadOnlyList<string>

Metodos:

AdicionarEspecialidade(string) : void (sem duplicatas)

Metodos sobrescritos:

ObterTipo() -> "Bibliotecario-Chefe" ou "Bibliotecario"

ObterResumo() -> adiciona CRB e marcador de chefe

ObterPermissoes() -> estende com permissoes admin/relatorios;
se ChefeDivisao, adiciona config do sistema

Observacao: construtor define Cargo = "Bibliotecario" via base(...)

CLASSE: Aluno (herda Pessoa diretamente)

Arquivo: Models/Aluno.cs

Propriedades adicionais:

Matricula : string (uppercase, trim)

Curso : string

Email : string

Telefone : string

AnoIngresso : int (entre 2000 e ano atual)

Ativo : bool

EmprestimosAtivos: int

Constante:

MaxEmprestimos : const int = 3

Propriedade computada:

PodePegarEmprestimo : bool (Ativo && EmprestimosAtivos < MaxEmprestimos)

Metodos sobrescritos:

ObterTipo() -> "Aluno"

ObterResumo() -> adiciona matricula, curso, contagem de emprestimos, status

4. HIERARQUIA DE CLASSES — ITENS DA BIBLIOTECA

ItemBiblioteca (abstrata)

└─ Livro

CLASSE ABSTRATA: ItemBiblioteca

Arquivo: Models/ItemBiblioteca.cs

Propriedades:

Id : int (> 0)

Titulo : string (nao vazio, trim)

AnoPublicacao : int (entre 1400 e ano atual)

Disponivel : bool

QuantidadeTotal : int

QuantidadeDisponivel : int

Metodos abstratos:

ObterCategoria() : string

ObterDetalhes() : string

Metodos concretos:

RealizarEmprestimo() : bool (decrementa QuantidadeDisponivel; false se esgotado)

RealizarDevolucao() : void (incrementa QuantidadeDisponivel, Disponivel = true)

ToString() : "[Categoria] ID: X | Titulo (Ano) | Disponivel: X/Y"

CLASSE: Livro (herda ItemBiblioteca)

Arquivo: Models/Livro.cs

Propriedades adicionais:

Autor : string

Editora : string

Genero : string

NumeroPaginas : int

Isbn : string (10 ou 13 digitos; strip '-' e ' ')

Metodos sobrescritos:

ObterCategoria() -> "Livro"

ObterDetalhes() -> string formatada multi-linha com todos os dados

5. ENTIDADE: Emprestimo

Arquivo: Models/Emprestimo.cs

ENUM: StatusEmprestimo

Ativo | Devolvido | Atrasado | Renovado

CLASSE: Emprestimo

Constantes:

DiasEmprestimoPadrao : static readonly int = 14

MaxRenovacoes : const int = 2

Propriedades:

Id : int

AlunoId : int

AlunoNome : string (desnormalizado)

LivroId : int

LivroTitulo : string (desnormalizado)

DataEmprestimo : DateTime

DataPrevistaDevolucao : DateTime (DataEmprestimo + 14 dias)

DataDevolucaoReal : DateTime? (nullable)

Status : StatusEmprestimo

NumeroRenovacoes : int

Propriedades computadas:

Atrasado : bool (Status == Ativo && Hoje > DataPrevistaDevolucao)

DiasAtraso : int (dias apos vencimento; 0 se nao atrasado)

MultaPorAtraso : decimal (DiasAtraso * R\$0,50)

Metodos:

Renovar() : bool (estende prazo em 14 dias; falha se NumeroRenovacoes >= 2
ou status nao ativo)

Devolver() : void (DataDevolucaoReal = Now; Status = Devolvido)

AtualizarStatus() : void (marca como Atrasado se vencido)

ToString() : override (string rica com aviso de atraso/multa)

6. INTERFACE GENERICA: IRepository<T>

Arquivo: Interfaces/IRepository.cs

Namespace: SistemaBiblioteca.Interfaces

Metodos:

void Adicionar(T entidade)

T BuscarPorId(int id)

List<T> BuscarTodos()

void Atualizar(T entidade)

void Remover(int id)

bool Existe(int id)

Observacao: interface definida mas nao formalmente implementada na versao atual. MenuPrincipal usa List<T> diretamente como repositario em memoria.

7. UTILITARIO: Validador (static)

Arquivo: Utils/Validador.cs

Namespace: SistemaBiblioteca.Utils

Todos os metodos de leitura loopam ate o usuario fornecer entrada valida.

Metodos de entrada:

LerInteiroPositivo(mensagem) -> int

LerInteiro(mensagem, min, max) -> int

LerDecimal(mensagem) -> decimal

LerStringObrigatoria(mensagem, minLen)-> string

LerData(mensagem) -> DateTime (dia/mes/ano separados)

LerEmail(mensagem) -> string

LerCpf(mensagem) -> string

ConfirmarOperacao(mensagem) -> bool (S/N)

Metodos de saida:

PausarTela() -> "Pressione qualquer tecla para continuar..."

ExibirSucesso(mensagem) -> "[OK] ..."

ExibirErro(mensagem) -> "[ERRO] ..."

ExibirAviso(mensagem) -> "[AVISO] ..."

ExibirInfo(mensagem) -> "[INFO] ..."

8. CONTROLADOR: MenuPrincipal

Arquivo: Menu/MenuPrincipal.cs

Namespace: SistemaBiblioteca.Menus

Tamanho: ~910 linhas

Repositorios em memoria (campos privados):

_alunos : readonly List<Aluno>

_livros : readonly List<Livro>

_emprestimos : readonly List<Emprestimo>

Contadores de ID:

_proximoldAluno, _proximoldLivro, _proximoldEmprestimo : int

Dados iniciais (seeding no construtor):

InicializarAlunos() -> 3 alunos (Ana Clara, Bruno Lima, Carla Mendes)

InicializarLivros() -> 5 livros (Clean Code, O Algoritmo da Vida,

Design Patterns, Estruturas de Dados,

Inteligencia Artificial)

Estrutura de menus:

Iniciar() -> loop principal; exibe menu e roteia

MenuAlunos() -> sub-menu alunos (6 opcoes)

MenuLivros() -> sub-menu livros (8 opcoes)

MenuEmprestimos() -> sub-menu emprestimos (7 opcoes)

MenuRelatorios() -> dashboard com contagens

ExibirPermissoesFuncionarios()-> demonstracao de polimorfismo ao vivo

Logica de negocio — Alunos:

CadastrarAluno() -> valida unicidade de CPF e Matricula

ListarAlunos() -> imprime todos via ToString()

BuscarAlunoPorId() -> busca linear por ID

BuscarAlunoPorNome() -> busca parcial case-insensitive (IndexOfOrdinalIgnoreCase)

AtualizarAluno() -> campos opcionais (Enter para pular)

RemoverAluno() -> bloqueado se ha emprestimos ativos; confirmacao S/N

Logica de negocio — Livros:

CadastrarLivro() -> valida unicidade de ISBN

ListarLivros(somenteDisp.) -> listagem com filtro opcional

BuscarLivroPorTitulo() -> busca parcial case-insensitive

BuscarLivroPorAutor() -> busca parcial case-insensitive

VerDetalhesLivro() -> chama ObterCategoria() e ObterDetalhes()

AtualizarLivro() -> campos opcionais

RemoverLivro() -> bloqueado se ha copias emprestadas; confirmacao

Logica de negocio — Empréstimos:

NovoEmprestimo() -> valida atividade do aluno, limite de empréstimos,
empréstimo duplicado e disponibilidade do livro;
chama livro.RealizarEmprestimo() e incrementa
aluno.EmprestimosAtivos

DevolverLivro() -> exibe aviso de atraso e multa; chama emp.Devolver(),
livro.RealizarDevolucao(), decrementa EmpréstimosAtivos

RenovarEmprestimo() -> chama emp.Renovar() (respeita MaxRenovacoes)

ListarEmprestimos() -> chama AtualizarStatus() em todos antes de exibir

EmprestimosDoAluno() -> filtra por Alunold

EmprestimosAtrasados()-> filtra atrasados e soma multas totais

9. REGRAS DE NEGOCIO

Emprestimos:

- Limite por aluno : 3 (Aluno.MaxEmprestimos)
- Prazo padrao : 14 dias (Emprestimo.DiasEmprestimoPadrao)
- Renovacoes permitidas : 2 (Emprestimo.MaxRenovacoes)
- Multa por atraso : R\$0,50 por dia

Validacoes de entrada:

- CPF : exatamente 11 digitos numericos
- Email : deve conter '@' e '.'
- Telefone : 10 ou 11 digitos numericos
- ISBN : 10 ou 13 digitos numericos
- Nome : minimo 3 caracteres
- AnoIngresso : entre 2000 e ano atual
- AnoPublicacao: entre 1400 e ano atual
- DataNascimento: entre 01/01/1900 e hoje
- Salario : >= 0

Restricoes de remocao:

- Aluno : nao pode ser removido se tiver emprestimos ativos

- Livro : nao pode ser removido se houver copias emprestadas

10. PADROES DE PROJETO E CONCEITOS DE POO UTILIZADOS

Heranca multinivel

Pessoa -> Usuario -> Funcionario -> Bibliotecario (4 niveis)

Classes abstratas

Pessoa e ItemBiblioteca nao podem ser instanciadas diretamente.

Template Method (parcial)

Pessoa.ObterResumo() chama o metodo abstrato ObterTipo() definido nas subclasses.

Interface generica

IRepository<T> — contrato de repositorio parametrizado por entidade.

Encapsulamento com validacao

Todos os modelos usam campos privados com setters que lancam
ArgumentException em caso de valor invalido.

Polimorfismo

List<Pessoa>, List<Funcionario> e ItemBiblioteca item = _livros[0]
permitem despacho em tempo de execucao para metodos sobrescritos.

Sobrescrita de metodos

ObterTipo(), ObterResumo() e ObterPermissoes() sobrescritos em cada nivel.

Propriedades computadas (sem backing field)

CpfFormatado, Idade, AnosServico, TelefoneFormatado,
PodePegarEmprestimo, Atrasado, DiasAtraso, MultaPorAtraso.

Enum

StatusEmprestimo: Ativo, Devolvido, Atrasado, Renovado.

Classe utilitaria estatica

Validador — agrupa todos os helpers de I/O sem necessidade de instancia.

Programacao defensiva

Todos os construtores e setters validam entradas com mensagens descritivas.

Repositorio em memoria

MenuPrincipal com List<T> simula persistencia sem banco de dados.

IDs auto-incremento

_proximoldAluno, _proximoldLivro, _proximoldEmprestimo.

Colecao somente leitura

Bibliotecario.Especialidades expoe IReadOnlyList<string> para
evitar mutacao externa da lista interna.

Desnormalizacao

Emprestimo armazena AlunoNome e LivroTitulo para preservar informacao

de exibicao mesmo que os registros originais sejam alterados.

Verificacao de tipo (pattern matching)

ExibirPermissoesFuncionarios() usa

"if (x is IInterface y)" para chamadas de metodo type-safe em tempo de execucao.

Constantes de regras de negocio

Aluno.MaxEmprestimos = 3

Emprestimo.MaxRenovacoes = 2

Emprestimo.DiasEmprestimoPadrao = 14

=====
=====

FIM DO DOCUMENTO

=====
=====